Episode 2: Mistakes, Errors, Scenarios and Interview Answers

Common wrong ideas, real error messages and fixes, scenario questions, and solid interview answers with diagrams

Part A: Common beginner mistakes

Mistake	What happens	Correct way
Writing `-p 80:8080` (reversed)	The page does not load: host 80 goes to a container port nothing listens on.	`-p HOST:CONTAINER`, for nginx `-p 8080:80`.
Forgetting `-d`	The terminal is stuck showing logs.	Add `-d`, or open another terminal. Ctrl+C usually stops a foreground container.
Putting options after the image name	They are passed to the app inside, not to Docker.	Options first: `docker run -d -p 8080:80 nginx`.
Using `docker ps` and thinking the container vanished	docker ps shows only RUNNING containers.	Use `docker ps -a` to see exited ones.
Confusing `rm` and `rmi`	Removing the wrong thing, or an error saying the image is in use.	rm = container, rmi = image. Remove containers first.
Trying to `rm` a running container	Docker refuses.	`docker stop web` first, or `docker rm -f web`.
Running `docker run` again and again	Each run creates a NEW container (and name clashes).	To restart an old one use `docker start NAME`.
Changing things inside with exec and expecting them to stay	The changes vanish when the container is removed.	Change the image (Dockerfile, next episode) or use volumes (Episode 4).
Using `latest` everywhere	You may get different versions at different times.	Pin versions like `nginx:1.27` for repeatable results.

Part B: Errors you will meet, and the fix

What you see	What it means	What to do
`Cannot connect to the Docker daemon ... Is the docker daemon running?`	The daemon is not running.	Start Docker Desktop, or on Linux start the Docker service.
`error during connect ... pipe/dockerDesktopLinuxEngine: The system cannot find the file specified` (Windows)	Docker Desktop is not running yet.	Open Docker Desktop and wait for the engine to be running.
`permission denied while trying to connect to the Docker daemon socket`	Your Linux user may not use Docker.	Add yourself to the docker group, log out and in, or use sudo.
`Conflict. The container name "/web" is already in use`	A container named web already exists (maybe exited).	`docker rm web` or choose another name.
`Bind for 0.0.0.0:8080 failed: port is already allocated`	Something already uses host port 8080.	Use another host port (`-p 8081:80`) or stop the other container.
`pull access denied ... repository does not exist`	Typo in the image name, such as ngnix.	Check the spelling on Docker Hub.
`Error response from daemon: No such container: web`	Wrong name, or it was already removed.	Check names with `docker ps -a`.
`cannot remove container ... container is running`	You tried rm on a running container.	Stop it first, or use `rm -f`.
`container ... is not running` (on exec)	exec works only on RUNNING containers.	`docker start web` first, or check why it exited with logs.
`OCI runtime exec failed ... executable file not found in $PATH`	That image has no bash.	Try `docker exec -it web sh`.
Browser: "This site can't be reached" on localhost:8080	Container stopped, wrong port mapping, or the app is not listening.	Check `docker ps` (PORTS column), then `docker logs`.

Debug in this order: 1) `docker ps -a` (is it running?) 2) `docker logs NAME` (what did it say?) 3) `docker inspect NAME` (ports, state, exit code) 4) `docker exec -it NAME sh` (look inside).

Part C: Scenario-based questions

Think first, then check the answer key in Part D.

Scenario 1: The page will not open

You ran `docker run -d -p 80:8080 --name web nginx` and localhost:8080 shows nothing. **What is wrong and how do you fix it?**

Your answer: _____

Scenario 2: It exits immediately

You run `docker run ubuntu` and docker ps shows nothing. **Where is the container and why did it stop?**

Your answer: _____

Scenario 3: Name already in use

Running your command again gives "container name /web is already in use". **Give two ways to fix it.**

Your answer: _____

Scenario 4: Port clash

A teammate's app already uses port 8080 on the shared server. **How do you run your nginx at the same time?**

Your answer: _____

Scenario 5: No logs on screen

You started a container with -d and want to see what the app is printing right now. **Which command and flag?**

Your answer: _____

Scenario 6: Cannot remove

`docker rm web` says the container is running. **What are your options?**

Your answer: _____

Scenario 7: Look inside

A container is running but behaves oddly. **How do you get a shell inside, and what if bash is missing?**

Your answer: _____

Scenario 8: Disk is filling up

After weeks of experiments the laptop has many exited containers and old images. **How do you clean up safely?**

Your answer: _____

Scenario 9: Cannot connect to daemon

A new teammate gets "Cannot connect to the Docker daemon" right after installing. **What do you check first on Windows/Mac and on Linux?**

Your answer: _____

Scenario 10: Which command?

Match the goal to the command: (a) see all containers including stopped, (b) follow logs live, (c) start a stopped container again, (d) see which host port is mapped.

Your answer: _____

Quick true or false

1. In -p 8080:80, 8080 is the port inside the container. True / False
2. docker ps shows stopped containers by default. True / False
3. docker run creates a new container every time. True / False
4. docker rmi removes a container. True / False
5. exec works on stopped containers. True / False
6. The docker CLI and the Docker daemon are the same thing. True / False

Part D: Answer key

- **Scenario 1:** The mapping is reversed. nginx listens on port 80 inside, so use `-p 8080:80`. Remove the old one (`docker rm -f web`) and run again.
- **Scenario 2:** `docker ps -a` shows it as Exited (0). The ubuntu image has no long-running program, so the container ends as soon as its main process finishes. Use `docker run -it ubuntu bash` for an interactive shell.
- **Scenario 3:** (1) Remove the old container: `docker rm web` (or rm -f). (2) Choose a different name, or reuse the old one with `docker start web`.
- **Scenario 4:** Map a different host port: `docker run -d -p 8081:80 --name web2 nginx`. The container port 80 can stay the same.
- **Scenario 5:** `docker logs -f web` follows the output live. Add `--tail 50` to start from the last 50 lines.
- **Scenario 6:** Stop it first (`docker stop web`) then `docker rm web`, or use `docker rm -f web` to do both.
- **Scenario 7:** `docker exec -it NAME bash`. If you get "executable file not found", use `sh`. Check `docker logs` as well.
- **Scenario 8:** List first with `docker ps -a` and `docker images`. Remove with `docker container prune` (stopped containers) and `docker image prune` (unused dangling images). Read the confirmation prompt before saying yes.
- **Scenario 9:** Windows/Mac: make sure Docker Desktop is open and the engine shows as running. Linux: check the Docker service is active; if it says permission denied, add your user to the docker group and log in again. Also try `docker info`.
- **Scenario 10:** (a) `docker ps -a` (b) `docker logs -f NAME` (c) `docker start NAME` (d) the PORTS column of `docker ps`, or `docker port NAME`.

True or false answers

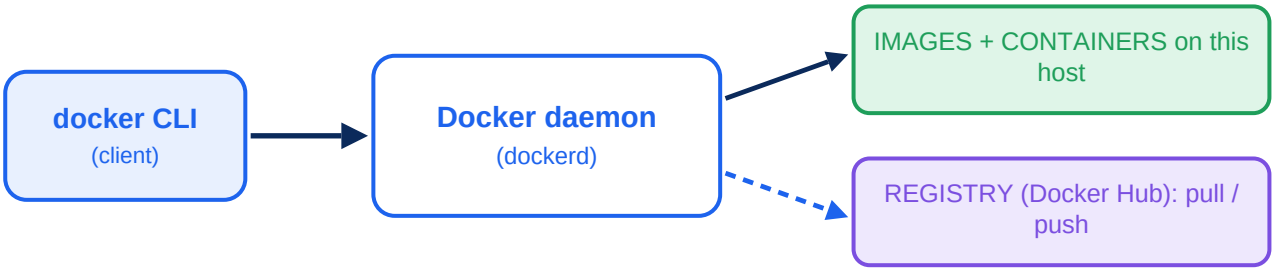
1. **False** (8080 is the host port) | 2. **False** (use -a) | 3. **True** | 4. **False** (rmi removes images) | 5. **False** (needs a running container) | 6. **False** (client vs background service)

Part E: Interview Corner - solid answers with diagrams

Use these as a base and say them in your own words. Real examples from your own lab make answers stronger.

> **Answer formula: 1. Define** in one sentence. **2. Explain how** it works. **3. Give an example** command or case. **4. Mention a trade-off or common pitfall.**

Interview Q1. Explain Docker's architecture: client, daemon, registry.

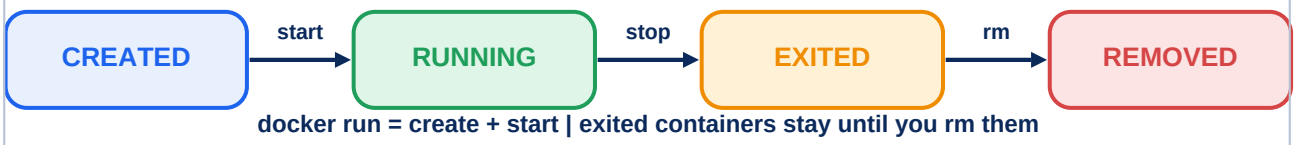


Say this first (30 sec)	Docker uses a **client-server** model. The **docker CLI** is the client. It sends requests over a REST API to the **Docker daemon (dockerd)**, which builds, runs and manages images and containers. Images are pulled from and pushed to a **registry** such as Docker Hub.
Add depth	The daemon and CLI can be on different machines (for example a remote host), which is why "cannot connect to the daemon" errors exist. Docker Desktop bundles the CLI, daemon and tools. Underneath, container runtimes such as containerd and runc do the low-level work.
Example to adapt	"When docker commands failed on a new laptop, I ran docker info to check whether the daemon was reachable."
Likely follow-up	"Who has access to the daemon?" Anyone who can talk to the Docker socket effectively has root-level power on the host, so the docker group must be limited.

Interview Q2. What happens when you run `docker run -d -p 8080:80 --name web nginx`?

Say this first (30 sec)	The CLI asks the daemon to run a container. The daemon looks for **nginx** locally, pulls it from the registry if it is missing, **creates** a container named web, sets up the port mapping 8080 to 80, **starts** it in the background (-d) and prints the container ID.
Add depth	Because no tag was given, Docker uses `nginx:latest`. The container runs the image's default command (nginx in the foreground). While that main process runs, the container stays Up.
Example to adapt	"I verify it with docker ps for status and ports, and curl localhost:8080 for the response."
Likely follow-up	"What if you run the same command again?" It fails because the name web is taken, and the port 8080 would clash too.

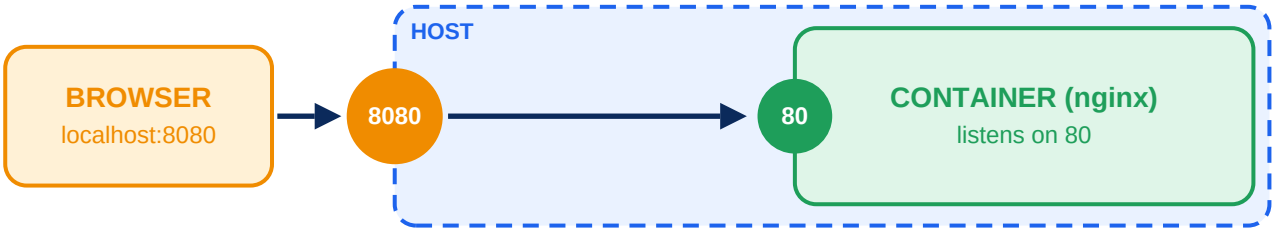
Interview Q3. Difference between `docker run`, `start`, `create` and exec?



docker run = create + start | exited containers stay until you rm them

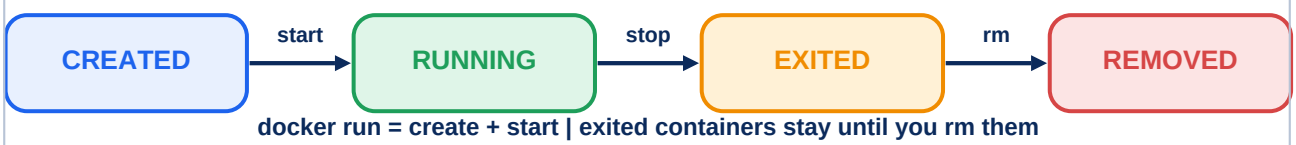
Say this first (30 sec)	**create** makes a container without starting it. **start** starts an existing stopped container. **run** is create plus start for a **new** container from an image. **exec** runs an extra command inside an **already running** container.
Add depth	Running run repeatedly creates many containers, which is a common source of clutter and name or port clashes. Use start/restart to bring an old one back, and exec only to inspect or debug.
Example to adapt	"To restart my web container after a config change I used docker restart web, not another run."
Likely follow-up	"Does start keep the old data?" Yes, a stopped container keeps its writable layer until it is removed.

Interview Q4. Explain port mapping. What does `-p 8080:80` mean?



Say this first (30 sec)	It **publishes** a container port on the host. The format is **HOST:CONTAINER**. Traffic to port 8080 on the host is forwarded to port 80 inside the container.
Add depth	Containers have an isolated network, so ports are not reachable from outside unless published. Two containers can both listen on 80 inside as long as their host ports differ (8080 and 8081). The host port must be free, or you get "port is already allocated". EXPOSE in a Dockerfile only documents a port; **-p** actually publishes it. Also, the app must listen on 0.0.0.0 inside the container, not only on 127.0.0.1.
Example to adapt	"I ran two copies of the same site on 8080 and 8081 to compare versions."
Likely follow-up	"What does -P (capital) do?" It publishes all exposed ports to random host ports.

Interview Q5. A container exits immediately. How do you troubleshoot?



docker run = create + start | exited containers stay until you rm them

Say this first (30 sec)	Start with `docker ps -a` to see the status and **exit code**, then `docker logs NAME` to see why. A container runs only while its main process runs, so if that process finishes or crashes, the container exits.

Add depth	Common causes: no long-running process (for example the ubuntu image), a crash from missing config, or a wrong command. Exit code 0 means normal finish, 1 usually an application error, 137 means it was killed (SIGKILL, often out of memory or a forced stop), 143 means SIGTERM. `docker inspect` shows the state details. Run it interactively with `-it` and a shell to experiment.
Example to adapt	"Exit code 137 with a memory limit set told me the container was being killed for using too much memory."
Likely follow-up	"How do you keep a container running just for testing?" Run a shell with -it, or a long-running command, never an infinite loop hack in production.

Interview Q6. `docker stop` vs `kill` vs `rm -f`?	
Say this first (30 sec)	**stop** sends SIGTERM so the app can shut down cleanly, then SIGKILL after a grace period (10 seconds by default). **kill** sends a signal immediately (SIGKILL by default). **rm -f** forcibly stops and deletes the container.
Add depth	Prefer stop so databases and servers can finish writes and close connections. The app must handle SIGTERM properly. The grace period can be changed with `-t`.
Example to adapt	"For a database container I always use stop, never kill."
Likely follow-up	"Why does stop sometimes take 10 seconds?" The app ignores SIGTERM, so Docker waits then force-kills it.

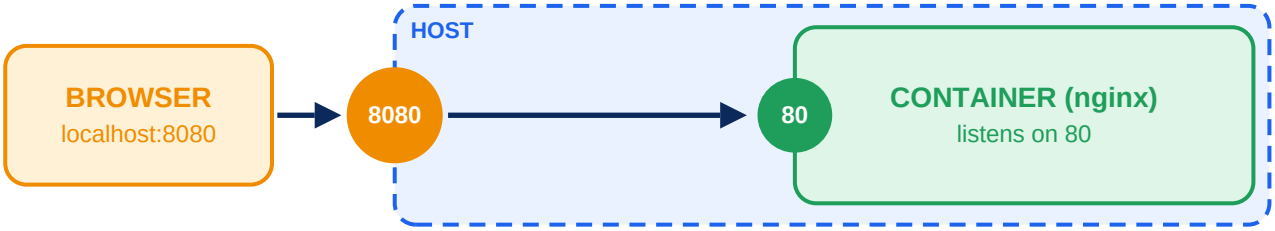
Interview Q7. `docker rm` vs `docker rmi`?	
Say this first (30 sec)	**rm** deletes a **container**. **rmi** deletes an **image**. An image cannot be removed while containers (even stopped ones) still use it.
Add depth	Removing a container does not remove its image, so the next run is fast. To clean up many items use `docker container prune` and `docker image prune`.
Example to adapt	"I removed old exited containers first, then the old image versions."
Likely follow-up	"What do the prune commands delete?" Stopped containers, and dangling (untagged, unused) images respectively. Check the prompt, and note that docker system prune -a removes all unused images.

Interview Q8. How do you debug a running container?	
Say this first (30 sec)	Follow a short path: `docker ps` (state and ports), `docker logs -f` (output), `docker inspect` (configuration, IP, mounts, exit code) and `docker exec -it NAME sh` (look inside).
Add depth	Useful extras: `docker stats` for live CPU and memory, `docker top NAME` for processes, and `docker port NAME` for mappings. exec changes are temporary: fix the cause in the image or configuration, not by hand in the container.
Example to adapt	"I found a wrong environment variable with docker inspect and fixed it in the run command."
Likely follow-up	"Why not install tools inside the container with exec?" They vanish when the container is replaced, and it makes the environment non-reproducible.

Interview Q9. What do `-i`, `-t` and `-d` mean?	
Say this first (30 sec)	**-i** keeps standard input open, **-t** allocates a pseudo-terminal, together **-it** gives an interactive shell. **-d** runs the container **detached** in the background.
Add depth	Interactive (-it) is for shells and tools you type into. Detached (-d) is for servers. They are normally not used together on the same web server container.
Example to adapt	"docker run -it --rm ubuntu bash gives me a throwaway Linux shell."
Likely follow-up	"What does --rm add?" It deletes the container automatically when it exits.

Interview Q10. SCENARIO: "port is already allocated" on 8080. What do you do?	
Say this first (30 sec)	Find what is using the port, then either free it or choose another host port. Run `docker ps` to check whether another container publishes 8080. If not, check the host (for example `ss -ltnp` on Linux or `lsof -i :8080` on macOS).
Add depth	Fix: stop the other container or process, or map to a free port such as `-p 8081:80`. Only one process can listen on a host port at a time. For many services behind one port, use a reverse proxy.
Example to adapt	"I moved my test copy to 8081 so it ran beside the existing one."
Likely follow-up	"Does changing the host port change the container?" No, the container still listens on 80 inside.

Interview Q11. SCENARIO: The container is Up but localhost:8080 does not load. Why?



Say this first (30 sec)	Check from the outside in. **1)** `docker ps`: is the mapping `0.0.0.0:8080->80/tcp` present and in the right order? **2)** `docker logs`: did the app start and which port is it listening on? **3)** Is the app bound to 0.0.0.0 rather than 127.0.0.1 inside the container? **4)** Firewall or another process on the host?
Add depth	A common cause is the app listening on a different port than the one mapped, or only on 127.0.0.1 inside the container, which cannot be reached through the published port. You can test from inside with `docker exec` and curl if it is installed.
Example to adapt	"The app listened on 3000 but I had mapped to 80. Mapping 8080:3000 fixed it."
Likely follow-up	"How can you confirm the app answers inside the container?" Exec in and request localhost with curl or wget.

Interview Q12. How do you clean up Docker safely on a busy machine?	
Say this first (30 sec)	List first (`docker ps -a`, `docker images`), then remove what you do not need. `docker container prune` removes stopped containers and `docker image prune` removes dangling images. Read each confirmation prompt.

Add depth	`docker system prune` is broader (also unused networks and build cache), and `-a` removes all unused images. Avoid it on shared servers unless you are sure, and never prune volumes blindly because that is where persistent data lives (Episode 4).
Example to adapt	"I scheduled a weekly image prune on build agents to save disk."
Likely follow-up	"What is the risk of prune on production?" Deleting images or data you still need, causing slow re-pulls or data loss.

Interview Q13. Explain Docker to someone who types the commands for the first time.	
Say this first (30 sec)	"You type commands to a tool called the Docker CLI. It sends them to a background service, the daemon, which downloads packaged apps (images) and runs them as isolated containers. The main commands are pull to get an image, run to start it, ps to see what is running, logs to read its output, exec to step inside, and stop and rm to clean up."
Add depth	Add: "-d runs it in the background and -p connects a port on your computer to the container."
Example to adapt	Use the waiter and chef story from the Story PDF.
Likely follow-up	"Which command do you use most?" Honest answer: ps, logs and exec for checking, run for starting.

Rapid-fire round (one-line answers)

Question	One-line answer
Client or daemon: which does the work?	The daemon.
Which flag runs in the background?	-d
Order in -p?	HOST:CONTAINER
Show stopped containers too?	docker ps -a
Read live logs?	docker logs -f NAME
Shell into a running container?	docker exec -it NAME bash (or sh)
Remove a running container in one go?	docker rm -f NAME
Remove an image?	docker rmi IMAGE
Does run create a new container each time?	Yes.
Why did my container exit at once?	Its main process finished or crashed. Check logs and exit code.

Practice tip: Draw the architecture and port-mapping diagrams on paper while you explain them out loud. Then do the 10-minute lab again without notes.